Vehicle Rental SaaS - Backend Build Prompt

Production-grade Go + Chi backend specification with Docker, PostgreSQL, Redis, queues, security and anti-vibe-coding rules

OBJECTIVE

Build a production-grade multi-tenant backend for a SaaS platform used by businesses that rent cars, bikes and scooters. The backend must power vehicle management, availability, reservations, customers, pickup/return, payments, deposits, contracts, maintenance, expenses, staff, reports, notifications, online booking and subscriptions.

CRITICAL RULE: DO NOT VIBE CODE. Inspect the repository first. Design the domain and boundaries before implementation. Keep handlers thin, business logic testable, database access isolated and security enforced server-side.

BACKEND STACK

Use Go, Chi router, PostgreSQL, SQLC, Redis, Asynq, Docker, Docker Compose and Caddy. Build a versioned REST API with OpenAPI documentation. Use Go's standard library where practical instead of unnecessary frameworks.

Recommended supporting packages:
- go-playground/validator for request validation
- pgx for PostgreSQL connectivity
- golang-migrate for migrations
- structured logging with log/slog
- UUIDs for public identifiers where appropriate
- bcrypt/argon2id for password hashing
- OpenAPI generation/documentation

SERVICE ARCHITECTURE

Use:
cmd/api/main.go
cmd/worker/main.go
internal/auth
internal/tenant
internal/user
internal/vehicle
internal/customer
internal/booking
internal/payment
internal/contract
internal/maintenance
internal/expense
internal/notification
internal/report
internal/marketing
internal/subscription
internal/integration

```
db/migrations
db/queries
db/generated
api/openapi
pkg/config
pkg/logger
pkg/httpx
pkg/pagination
pkg/storage
```

Keep domain logic inside features. Do not create a huge generic service package containing unrelated logic.

REQUEST FLOW

Use a predictable flow:

HTTP Request
-> middleware
-> router
-> handler
-> request validation
-> service/use-case
-> repository/SQLC
-> PostgreSQL/Redis
-> response mapper
-> JSON response

Handlers should not contain complex business rules or SQL.

DOCKER SETUP

Create containers/services for:
- api: Go REST API
- worker: Asynq background worker
- postgres: PostgreSQL
- redis: Redis
- caddy: reverse proxy and HTTPS
- optional migration job/container

Use multi-stage Docker builds, small runtime images, non-root runtime where practical, health checks, environment variables and persistent volumes. Provide docker-compose.yml for local development.

Do not store production secrets in the repository.

MULTI-TENANCY

The SaaS is multi-tenant. Every company/rental business is a tenant.

A protected request must resolve the authenticated user and tenant from trusted authentication/session context. Never trust a company_id supplied by the browser as the authority.

Every tenant-owned query must scope by tenant/company ID.

Implement:
- tenant/company model
- tenant membership
- tenant-aware repositories
- tenant-aware authorization
- tenant-scoped unique constraints
- tenant-safe background jobs
- tenant-safe file/object paths
- tenant-safe reporting

A user from Company A must never be able to read or mutate Company B data.

AUTHENTICATION + AUTHORIZATION

Implement:
- registration/invitation flow
- login
- logout
- refresh/session handling
- email verification
- forgot password
- reset password
- optional 2FA
- session/device management

Authorization:
- RBAC roles
- granular permissions
- middleware/policy checks
- permission checks in services for sensitive operations

Never rely only on frontend permission checks.

CORE DOMAIN MODULES

Implement modules for:

```
Auth
Tenant / Company
Locations
Users / Staff
Roles / Permissions
Vehicles
Vehicle Categories
Vehicle Documents
Vehicle Availability
Pricing
Customers
Identity / License Verification
Bookings
Pickup / Return
Rental Contracts
Payments
Deposits
Refunds
Invoices
Maintenance
Vehicle Damage
Expenses
Fuel
Notifications
Reports
Marketing
Coupons / Discounts
Subscriptions
Usage
Integrations
Audit Logs
```

VEHICLE DOMAIN

Vehicle must support:
- make/model
- registration/plate
- type/category
- location
- status
- pricing
- mileage/odometer
- fuel type
- transmission
- seats/capacity where relevant
- photos
- documents
- insurance/registration expiry
- maintenance schedule
- damage records
- availability

Statuses should be explicit and validated, not arbitrary strings scattered through the code.

BOOKING DOMAIN

Booking must support:
- reservation creation
- pending/confirmed/cancelled states
- pickup time
- return time
- vehicle
- customer
- location
- pricing breakdown
- taxes

- discounts
- deposit
- payment status
- contract status
- notes
- booking source
- extension
- pickup
- return
- cancellation
- audit history

CRITICAL: Prevent overlapping reservations for the same vehicle. This must be enforced server-side and with appropriate PostgreSQL constraints/locking/transaction strategy. Frontend availability checks are not sufficient.

PRICING ENGINE

Implement a centralized pricing service that can calculate:
- base rate
- daily/weekly/monthly rates
- seasonal rates
- location-specific rates
- duration discounts
- coupons
- taxes
- extras
- late fees
- damage charges
- deposit
- final total

Do not duplicate pricing calculations across booking, invoice and payment code. Use one canonical pricing calculation path.

PICKUP + RETURN

Pickup workflow:
- verify booking
- verify customer identity/license
- record odometer
- record fuel
- record pre-rental damage
- upload photos
- collect/sign agreement
- collect deposit/payment
- transition vehicle to rented

Return workflow:
- verify rental
- record return time
- odometer
- fuel
- new damage
- late fee
- extra charges
- deposit deduction/refund
- final invoice
- transition vehicle back to available or maintenance

PAYMENTS

Payment module must support:
- transaction records
- payment methods

- online gateways
- cash/bank records
- deposits
- refunds
- invoices
- reconciliation
- payment references
- idempotency

Every payment mutation must be safe against duplicate requests.

Webhook processing must be idempotent. Never trust a client-provided payment-success value. Verify gateway signatures and server-side status.

PAYMENT PROVIDERS

Design a provider abstraction so gateways can be added without rewriting booking/payment logic.

Potential providers:
- eSewa
- Khalti
- Stripe
- bank/manual transfer

Gateway-specific code belongs in integration/provider modules. The core payment service should work with a common provider interface.

DATABASE

Use PostgreSQL as the source of truth. Use migrations for schema changes.

Use SQLC for type-safe query generation.

```
Expected major tables/entities:
tenants
locations
users
tenant_memberships
roles
permissions
role_permissions
vehicles
vehicle_categories
vehicle_documents
vehicle_photos
vehicle_locations
vehicle_pricing
customers
customer_documents
customer_verifications
bookings
booking_items/extras where applicable
rental_checklists
rental_contracts
contract_templates
payments
payment_webhooks
deposits
refunds
invoices
maintenance_records
maintenance_work_orders
damage_records
expenses
expense_categories
fuel_records
notifications
notification_templates
coupons
discounts
subscriptions
subscription_events
audit_logs
```

Use foreign keys, indexes, check constraints, unique constraints and timestamps appropriately.

REDIS + BACKGROUND JOBS

Use Redis for:
- short-lived cache
- rate limiting where appropriate
- distributed coordination where needed
- job queues

Use Asynq workers for:
- emails
- SMS
- WhatsApp notifications
- payment reconciliation
- webhook processing
- report generation
- document processing
- reminder jobs
- expiry notifications
- scheduled cleanup

Jobs must be idempotent and safe to retry.

NOTIFICATIONS

Create a notification service supporting:
- in-app
- email
- SMS
- WhatsApp where configured

Use templates and event-driven triggers:
- booking created
- booking confirmed
- payment received
- pickup reminder
- return reminder
- overdue rental
- document expiry
- maintenance due
- refund processed

API DESIGN

Use a versioned REST API such as /api/v1.

Standardize:
- success response format
- error response format
- validation errors
- pagination
- filtering
- sorting
- search
- date range filters
- request IDs
- timestamps

Use HTTP status codes correctly. Never expose stack traces or internal database errors to clients.

API RESOURCE GROUPS

Implement route groups approximately as:

```
/api/v1/auth
/api/v1/tenants
/api/v1/users
/api/v1/roles
```

```
/api/v1/vehicles
/api/v1/vehicle-categories
/api/v1/vehicle-documents
/api/v1/customers
/api/v1/bookings
/api/v1/availability
/api/v1/pickups
/api/v1/returns
/api/v1/payments
/api/v1/deposits
/api/v1/refunds
/api/v1/invoices
/api/v1/contracts
/api/v1/maintenance
/api/v1/expenses
/api/v1/reports
/api/v1/notifications
/api/v1/coupons
/api/v1/subscription
/api/v1/settings
/api/v1/audit-logs
```

Do not implement every endpoint blindly. Define request/response models and authorization for each resource first.

SECURITY

```
Implement:
- TLS through Caddy in production
- secure authentication/session handling
- password hashing
- request validation
- rate limiting
- CORS policy
- secure headers
- authorization
- tenant isolation
- input/output validation
- SQL injection protection through parameterized SQL/SQLC
- file upload validation
- signed/private object URLs where appropriate
- webhook signature verification
- audit logging
- secret management through environment/configuration
```

Never trust client-side authorization.

FILE STORAGE

Use S3-compatible object storage for:
- vehicle photos
- customer identity/license documents
- rental damage photos
- signed contracts
- receipts
- invoices

The backend should issue controlled upload/download permissions. Do not expose private documents through publicly guessable URLs.

REPORTING

Create report services for:
- revenue
- bookings
- fleet utilization
- customers
- payments
- expenses
- profit and loss

Use optimized queries and indexes. Do not calculate large reports by loading every row into memory. For heavy reports, generate asynchronously and provide downloadable files.

AUDIT LOGGING

Audit sensitive actions:
- login/security changes
- user/role changes
- booking changes
- cancellation
- payment changes
- refunds
- deposit deductions
- contract signatures
- vehicle status changes
- maintenance
- expense changes
- settings changes

Store actor, tenant, action, target, timestamp and relevant metadata without leaking secrets.

OBSERVABILITY

Use structured logs with slog. Include request ID, tenant ID where safe, actor/user ID where appropriate, route, status and latency.

Add:
- health endpoint
- readiness endpoint
- liveness endpoint
- database connectivity checks
- Redis connectivity checks
- worker health/monitoring hooks

Do not log passwords, tokens, payment secrets or private document contents.

TESTING

Write:
- unit tests for business rules
- repository/integration tests for critical database behavior
- handler tests for important endpoints
- authorization tests
- tenant-isolation tests
- booking overlap tests
- pricing tests
- payment idempotency tests
- webhook verification tests

Critical rules must have tests before declaring them complete.

MIGRATIONS

Use versioned database migrations. Never edit production schema manually.

Each schema change must:
- have an ordered migration
- be reversible where practical
- include indexes/constraints
- be tested against existing data assumptions

CONFIGURATION

Centralize configuration loading and validation. Environment variables should configure:
database
redis
JWT/session secrets
object storage
email
SMS

WhatsApp
payment providers
frontend URL
allowed origins
log level
production/development mode

Fail fast when required production configuration is missing.

CLEAN CODE - STRICT ANTI-VIBE RULES

```
1. Inspect before coding.
2. Design domain boundaries before implementation.
3. Keep handlers thin.
4. Keep services/use-cases focused.
5. Keep SQL and repository access separated.
6. Use SQLC-generated types where appropriate.
7. Do not create giant files.
8. Do not duplicate business rules.
9. Do not use any/interface{} unnecessarily.
10. Do not ignore errors.
11. Do not suppress compiler/linter errors.
12. Do not hide authorization in the frontend.
13. Do not trust browser-supplied tenant IDs.
14. Do not hardcode secrets.
15. Do not expose internal errors.
16. Do not create fake endpoints.
17. Do not leave TODO endpoints for required features.
18. Keep database transactions around financial and state-transition operations.
19. Make retries and jobs idempotent.
20. Add tests for critical domain rules.
```

IMPLEMENTATION PHASES

Phase 1: repository inspection, Go module setup, configuration, logging, Docker, PostgreSQL, migrations, Redis, health checks and base HTTP server.

Phase 2: authentication, tenants, users, roles, permissions and audit logging.

Phase 3: vehicles, categories, documents, locations and availability.

Phase 4: customers, verification and bookings.

Phase 5: pickup, return, contracts, pricing and deposits.

Phase 6: payments, refunds, invoices and payment webhooks.

Phase 7: maintenance, expenses, notifications and background workers.

Phase 8: reports, marketing, subscription and integrations.

Phase 9: security hardening, performance, tests, OpenAPI, observability and production deployment.

DEFINITION OF DONE

```
Backend is complete only when the API builds cleanly, migrations apply from a fresh database, Docker services start reliably, all protected endpoints enforce authentication/authorization, tenant isolation is verified, critical booking/payment rules are transaction-safe, background jobs are retry-safe, webhooks are idempotent, OpenAPI is documented, tests cover critical business rules, logs are structured, health checks work, and the React frontend can consume the API without rewriting the backend architecture.
```

FINAL INSTRUCTION

START BY INSPECTING THE EXISTING BACKEND/REPOSITORY. DO NOT GENERATE THE WHOLE SYSTEM IN ONE UNCONTROLLED PASS. BUILD THE FOUNDATION FIRST, THEN IMPLEMENT THE PHASES IN ORDER. AFTER EACH PHASE, RUN GO TESTS, BUILD/LINT CHECKS, DATABASE MIGRATION CHECKS AND VERIFY THE API. FIX ERRORS BEFORE CONTINUING. DO NOT VIBE CODE.